PYTHON SET

BY SUJAL SHRESTHA

SET

A set in Python is an unordered collection of unique and mutable elements. Sets are commonly used to store elements where duplicates are not allowed, and their operations are optimized for checking membership, union, intersection, and difference.

Syntax:

```
myset={item1, item2, item3}
print(myset)
```

CREATING A SET

a. Using Curly Braces {}:

```
myset={1, 2, 4, 6 , 9}
print(myset)
```

b. Using set() Constructor:

```
myset=set([1, 2, 5, 7, 13])
```

c. Empty Set:

```
emptySet= set()   #Using {} creates an empty dictionary, not a set
```

SET OPERATIONS

a. Adding Elements:

```
my_set = {1, 2}
my_set.add(3)
print(my_set)
```

b. Using set() Constructor:

```
my_set = {1, 2, 3}
my_set.remove(2) # Removes 2; raises KeyError if 2 is not in the set
my_set.discard(4)
print(my_set)
```

SET OPERATIONS

c. Checking Memberhsip:

```
my_set = {1, 2, 3}
print(2 in my_set)
print(4 in my_set)
```

d. Set Union:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
union_set = set1 | set2
print(union_set)
```

SET OPERATIONS

e. Set Intersection:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
intersection_set = set1 & set2
```

f. Set Difference:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
difference_set = set1 - set2
```

SET OPERATIONS

e. Set Intersection:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
sym_diff_set = set1 ^ set2
```

ACCESSING ELEMENTS IN A SET

Sets are unordered collections, so individual elements cannot be accessed using indexing or slicing (like with lists or tuples). Instead, you can:

- Check membership with the in or not in operators.
- Iterate through the set to access all elements.

Example:

```
my_set = {1, 2, 3, 4}
print(2 in my_set)
print(5 in my_set)
```

ITERATING OVER A SET

```
my_set = {1, 2, 3, 4}
for element in my_set:
    print(element)
```

THANK YOU